Interfaces

Interface

- defines members, usually without implementing them
- all members are public and abstract by default
- access modifiers cannot be used explicitly
- cannot define fields

- a class can inherit from one or more interfaces and must implement all members
- no object can be created from an interface
- but an object can be converted to the data type of the interface

Interface

- is created by using the interface keyword
- Properties are not implemented automatically (even if it is the same structure)

```
public interface IVehicle
{
    string Description { get; set; }

    int MaxVelocity { get; set; }

    void Drive();
}

class Car : IVehicle
{
    public string Description { get; set; }

    public int MaxVelocity { get; set; }

    public void Drive()
    {
        Console.WriteLine("The car is driving.");
    }
}
```

Methoden und statische Properties

- Interfaces can have static members
- can only be called via the interface
- Static properties and all methods have a default implementation

```
public interface IVehicle
{
    static string Test = "A string";

    string Description { get; set; }

    int MaxVelocity { get; set; }

    void Drive();

    public string Info()
    {
        return "I am a " + Description;
    }
}

class Car : IVehicle
{
    public string Description { get; set; }

    public int MaxVelocity { get; set; }

    public void Drive()
    {
        Console.WriteLine("The car is driving.");
    }
}
```

implicit implementation

- Access to members directly via the object type

```
public interface IVehicle
{
    void Drive();
}

class Car : IVehicle
{
    public void Drive()
    {
        Console.WriteLine("The car is driving.");
    }
}

Car car = new Car();
car.Drive();
```

explicit implementation

- the members cannot use access modifiers
- Access only via conversion to the type of the interface

```
public interface IVehicle
{
    void Drive();
}

class Car : IVehicle
{
    void IVehicle.Drive()
    {
        Console.WriteLine("The car is driving.");
    }
}

Car car = new Car();
car.Drive(); //Not possible

IVehicle vehicle = car as IVehicle;
vehicle.Drive();
```